

CASE STUDY REPORT

CNN-Based Face Recognition System Implementation

Abstract

Face recognition has become one of the most important applications of computer vision and artificial intelligence. It is widely used in biometric authentication systems, surveillance systems, and security applications. With advances in deep learning, Convolutional Neural Networks (CNNs) have become the most effective approach for face recognition in images and video streams.

This project presents the implementation of a **CNN-based Face Recognition System using Python, OpenCV, and TensorFlow/Keras**. The system can collect facial images, train a CNN model on them, and perform real-time face recognition using a webcam. The implementation is divided into three main modules: **face data collection, CNN model training, and real-time face recognition**.

The project demonstrates how deep learning models can automatically extract facial features and classify individuals with high accuracy. It also highlights the importance of dataset preparation, model training, and real-time inference in building a functional face recognition system.

1. Introduction

Face recognition is a biometric technology used to identify or verify individuals based on their facial features. Unlike traditional authentication systems such as passwords or ID cards, face recognition relies on unique physical characteristics of a person's face.

Traditional face recognition systems relied on handcrafted feature extraction techniques such as **Eigenfaces, Fisherfaces, and Local Binary Patterns (LBP)**. Although these techniques were useful in controlled environments, they often struggled with variations in lighting, pose, and facial expressions.

With the emergence of **Deep Learning**, particularly **Convolutional Neural Networks (CNNs)**, face recognition systems have significantly improved in accuracy and robustness. CNN models can automatically learn important facial features from images without manual feature engineering.

In this project, a **CNN-based face recognition system** is implemented using Python. The system includes modules for collecting face images, training a CNN model, and recognizing faces in real-time using a webcam.

2. Objectives of the Project

The main objectives of this project are:

1. To design and implement a CNN-based face recognition system.
 2. To collect facial images and create a structured dataset.
 3. To train a CNN model capable of recognizing multiple individuals.
 4. To implement real-time face detection and recognition using a webcam.
 5. To demonstrate the working of deep learning in computer vision applications.
-

3. System Architecture

The face recognition system developed in this project consists of three main stages:

1. **Face Dataset Collection**
2. **CNN Model Training**
3. **Real-Time Face Recognition**

The workflow of the system is as follows:

1. Capture facial images using a webcam.
 2. Store the images in a structured dataset.
 3. Train a CNN model using the collected dataset.
 4. Detect faces from the webcam feed.
 5. Predict the identity of the detected face using the trained CNN model.
-

4. Dataset Structure

The dataset used in this project consists of images organized into folders where each folder represents a specific person.

Example dataset structure:

```
dataset
|
|— Laxman
|   |— 1.jpg
|   |— 2.jpg
|   |— 3.jpg
|
|— Person2
|   |— 1.jpg
|   |— 2.jpg
|
|— Person3
```

Each folder contains multiple images of the same person. This structure allows the CNN model to learn the facial features associated with each individual.

5. Face Dataset Collection (collect_faces.py)

The **collect_faces.py** script is responsible for capturing facial images from a webcam and storing them in the dataset folder.

Working of the Script

1. The user enters their name.
2. A folder with the user's name is created in the dataset directory.
3. The webcam captures video frames.
4. OpenCV detects faces in each frame using the **Haar Cascade face detection algorithm**.
5. The detected face is converted to grayscale and resized to a fixed size.
6. The processed face image is stored in the dataset folder.

Purpose

This script allows the system to easily add new individuals to the dataset without manually collecting images.

Key Features

- Real-time face detection
- Automatic dataset creation
- Image preprocessing
- Face cropping and resizing

This step is essential because CNN models require a sufficient number of images to learn the facial characteristics of each person.

6. CNN Model Training (train_model.py)

The **train_model.py** script is used to train a Convolutional Neural Network using the facial images stored in the dataset.

Steps Performed in Training

1. Load images from the dataset folders.
2. Convert images to grayscale format.
3. Resize images to a fixed resolution (100×100).
4. Normalize pixel values.
5. Encode labels corresponding to each person.
6. Train the CNN model using the prepared dataset.

CNN Architecture Used

The CNN model implemented in this project consists of the following layers:

1. **Input Layer**
 - Input size: 100 × 100 grayscale images
2. **Convolution Layer (32 filters)**
 - Extracts low-level features such as edges
3. **Max Pooling Layer**
 - Reduces feature map dimensions
4. **Convolution Layer (64 filters)**
 - Extracts more complex facial patterns
5. **Max Pooling Layer**
6. **Flatten Layer**
 - Converts 2D feature maps into a 1D vector
7. **Dense Layer (128 neurons)**
 - Learns higher-level facial representations
8. **Output Layer (Softmax)**
 - Classifies the face into one of the known identities

The trained model is saved as:

face_model.h5

This file contains the learned parameters of the CNN and is used during real-time recognition.

7. Real-Time Face Recognition (recognize_face.py)

The **recognize_face.py** script performs real-time face recognition using the trained CNN model.

Working of the Script

1. Load the trained CNN model.
2. Start the webcam.
3. Detect faces using Haar Cascade.
4. Extract the detected face region.
5. Preprocess the face image.
6. Pass the face image to the CNN model.
7. The model predicts the identity of the person.
8. Display the predicted name above the detected face.

Output

The system displays a bounding box around the detected face and prints the predicted name on the screen.

Example output:

[Laxman]

This enables the system to identify individuals in real time using the trained model.

8. Technologies Used

The following technologies were used to develop the system:

Programming Language

- Python

Libraries

- OpenCV (Face detection and image processing)
- TensorFlow / Keras (Deep learning model)
- NumPy (Numerical operations)
- Scikit-learn (Label encoding)

Tools

- Visual Studio Code
 - GitHub (Version control)
-

9. Applications

CNN-based face recognition systems have many real-world applications including:

Security Systems

Used in surveillance cameras and access control systems.

Smartphone Authentication

Used for unlocking smartphones using facial recognition.

Attendance Systems

Automated attendance tracking in schools and offices.

Banking and Identity Verification

Used for identity verification in financial institutions.

10. Challenges and Limitations

Despite its advantages, the system has several limitations.

Lighting Conditions

Poor lighting can reduce detection accuracy.

Pose Variations

Faces at extreme angles may not be detected properly.

Dataset Size

Small datasets can reduce model accuracy.

Unknown Face Detection

The system may classify unknown faces as known individuals if a confidence threshold is not implemented.

11. Future Improvements

The system can be further improved in several ways:

- Implementing **FaceNet or DeepFace** for higher accuracy
 - Adding **unknown face detection**
 - Using **larger datasets**
 - Creating a **web-based interface**
 - Storing recognition results in a database
-

12. Conclusion

This project successfully demonstrates the implementation of a **CNN-based Face Recognition System** using Python and deep learning libraries. The system can collect face images, train a CNN model, and recognize faces in real time using a webcam.

The implementation highlights how deep learning techniques can automatically learn facial features and perform accurate classification of individuals. Although the system has certain limitations, it provides a strong foundation for understanding how modern face recognition systems are developed.

With further improvements and larger datasets, CNN-based face recognition systems can be used in a wide range of real-world applications such as security, authentication, and surveillance.

References

1. Goodfellow, I., Bengio, Y., & Courville, A. *Deep Learning*. MIT Press.
2. OpenCV Documentation – <https://opencv.org>
3. TensorFlow/Keras Documentation – <https://tensorflow.org>
4. Labelled Faces in the Wild (LFW) Dataset
5. Face Recognition using Deep Learning Research Papers

GitHub Repo Link: <https://github.com/laxmanmudigonda/cnn-face-recognition>